UIDE

Escuela de Ciencias de la Computación
Ingeniería en Sistemas de Información

PROYECTO INTEGRADOR – TERCER NIVEL

Fase 1: Contexto, planificación y organización

Documento de contexto y selección del sistema

Proyecto	Plataforma empresarial distribuida – Chat Empresarial en Go
Integrantes	Santiago Yambay Daniel Villareal Esteban Trujillo
Docentes	Programación Orientada a Objetos: Reyes Chacon Ivan Galo / Salazar Torres María del Carmen Tecnologías de Comunicación en Red: Vinueza Bustamante Bryan Steven Fundamentos de Seguridad: Lopez Tito Juan Fernando Probabilidad y Estadística: Cajas Espinosa Edgar Tecnología y Sociedad: Pérez Jurado Marcelo Fernando
Periodo académico	Septiembre 2026 – Enero 2027
Fecha de entrega	12 de octubre de 2026

1. Descripción del problema y necesidad organizacional

Una empresa dedicada al desarrollo de soluciones tecnológicas abrió una segunda sede operativa (en adelante, **Sede A** y **Sede B**). Ambas sedes deben funcionar como una sola organización, pero el crecimiento introduce un problema concreto: **no existe un canal interno, controlado y verificable para que el personal de las dos sedes se comunique**.

1.1 Problemática identificada

- **Comunicación dispersa:** sin una plataforma institucional, las coordinaciones se tienden a repartir entre mensajería personal y correo, lo que fragmenta la información y dificulta recuperar acuerdos.
- **Identidad no verificable:** cualquier cuenta personal puede suplantar a un compañero; la empresa no controla quién accede ni con qué privilegios.
- **Información corporativa fuera del control de la empresa:** conversaciones de trabajo viajan por servicios externos sin políticas de acceso ni trazabilidad.
- **Sin evidencia cuantitativa:** la empresa no puede medir tiempos de respuesta, disponibilidad ni uso real del servicio para tomar decisiones.
- **Dependencia de la conectividad entre sedes:** la comunicación entre Sede A y Sede B debe sostenerse sobre la infraestructura de red propia (VLAN, OSPF, firewall) que el equipo diseñará.

1.2 Necesidad

La empresa requiere un **servicio de comunicación interna entre sedes, con autenticación de usuarios, historial básico y control de sesiones**, desplegado sobre su propia infraestructura y capaz de generar datos medibles que permitan evaluar su desempeño. Este servicio es el componente de software (Go) de la plataforma empresarial distribuida que integra además redes, seguridad, estadística y análisis socio-tecnológico.

2. Actores y usuarios iniciales

La identificación inicial de actores se detalla, junto con los roles del equipo, en la **Matriz de actores, roles y responsabilidades** (documento 02). Se resume aquí:

Actor	Tipo	Descripción	Interacción con el chat
Empleado	Usuario final	Personal de la Sede A o B que necesita comunicarse con compañeros.	Inicia sesión, envía y recibe mensajes, consulta su historial, cierra sesión.
Administrador del sistema	Usuario privilegiado	Responsable de gestionar cuentas y supervisar el servicio.	Gestiona usuarios en el servicio de autenticación; consulta métricas y sesiones activas.
Servicio de autenticación (FastAPI)	Sistema	Servicio independiente que registra usuarios y emite/valida sesiones.	Responde a las solicitudes de inicio de sesión y validación de token del servidor Go.
Servidor de chat (Go)	Sistema	Aplicación servidor que centraliza conexiones, mensajes e historial.	Valida la sesión de cada cliente, difunde mensajes y guarda el historial.
Administrador de red	Soporte	Mantiene VLAN, enrutamiento OSPF, DHCP, NAT y firewall.	Garantiza la conectividad y las políticas de acceso entre sedes.

Actor	Tipo	Descripción	Interacción con el chat
Docentes evaluadores	Externo	Revisan entregables y la defensa del proyecto.	Observan la demostración y verifican los requisitos.

3. Selección del sistema empresarial

3.1 Opciones evaluadas

Se evaluaron las cuatro opciones propuestas para la aplicación empresarial con los criterios exigidos por el proyecto: problema organizacional que resuelve, operaciones cliente-servidor reales y datos medibles para Estadística; además se consideró el riesgo de implementación.

Opción	Problema que resuelve	Operaciones cliente-servidor	Datos medibles	Riesgo / complejidad
Gestión de correos internos	Comunicación formal y asíncrona.	Media: operaciones tipo bandeja (listar, leer, borrar).	Media: volumen y tiempos de entrega.	Media: varios módulos (bandejas, estados).
Chat empresarial (elegida)	**Coordinación inmediata entre sedes.**	**Alta: conexiones concurrentes y mensajes en tiempo real.**	**Alta: latencia, mensajes/hora, sesiones, errores.**	Media: concurrencia en Go, manejable con goroutines.
Gestión de imágenes	Almacenar y clasificar recursos visuales.	Media: carga, consulta y descarga.	Media: tamaños y tiempos de transferencia.	Media: almacenamiento y metadatos.
Gestión de videos	Almacenar y compartir videos.	Media: carga y descarga de archivos grandes.	Media-baja: dominado por el ancho de banda.	Alta: archivos pesados sobre el enlace Wi-Fi compartido entre sedes.

Decisión: el equipo desarrollará el **Chat Empresarial**.

3.2 ¿Qué problema de la empresa resuelve?

El chat da a ambas sedes un **canal único y controlado** de comunicación inmediata. Reemplaza las conversaciones dispersas por un servicio institucional en el que cada mensaje pertenece a un usuario autenticado, se conserva un historial básico y el acceso se controla mediante sesiones. Además, al operar sobre la red propia de la empresa, la información no depende de servicios externos.

3.3 ¿Por qué permite operaciones cliente-servidor reales?

- El **servidor Go** y el **cliente Go** son procesos distintos que se ejecutan en equipos diferentes (un cliente en una sede consume lo que ejecuta el servidor en la otra, o en un segmento VLAN distinto), por lo que la comunicación atraviesa la red real y no una simulación en una sola máquina.
- Cada acción del usuario es una operación remota: **iniciar sesión, enviar mensaje, recibir mensaje, consultar historial y cerrar sesión**.
- El servidor atiende **varios clientes de forma concurrente** (goroutines), valida cada solicitud contra el servicio de autenticación FastAPI y persiste el historial.
- El acceso a los recursos depende de una **sesión válida emitida por FastAPI** (RF-SW-03), lo que obliga a una integración real entre componentes.

3.4 ¿Qué datos medibles generará para Estadística?

Cada operación del chat produce registros que alimentan el análisis descriptivo e inferencial de Fase 6. La población, la muestra y el instrumento definitivo se concretarán en la Fase 2; la propuesta inicial es:

Variable	Tipo	Cómo se mide	Unidad	Vinculada a
Latencia de entrega de un mensaje	Cuantitativa continua	Diferencia entre el instante de envío y el de recepción, registrada por la aplicación.	ms	RNF-01
Tiempo de respuesta del inicio de sesión	Cuantitativa continua	Tiempo entre la solicitud de login y la respuesta del servidor.	ms	RNF-01
Mensajes enviados por usuario, hora y sede	Cuantitativa discreta	Conteo en el registro (log) del servidor.	mensajes	RF-SW-02
Duración de sesión	Cuantitativa continua	Diferencia entre inicio y cierre/expiración de sesión.	min	RF-SW-03
Tasa de errores	Cuantitativa (proporción)	Operaciones fallidas / operaciones totales.	%	RNF-02
Disponibilidad del servicio	Cuantitativa (proporción)	Tiempo operativo / tiempo total de pruebas planificadas.	%	RNF-02
Percepción de usabilidad	Ordinal (escala Likert 1–5)	Cuestionario breve aplicado a usuarios de prueba.	puntaje	RNF-05

4. Alcance del componente de software

Para evitar ambigüedades, el alcance se separa en lo **mínimo obligatorio** (necesario para aprobar y cumplir los criterios de aceptación) y lo **opcional** (valor agregado, que solo se desarrollará si no compromete lo mínimo, tal como exige el proyecto).

Mínimo obligatorio	Opcional (valor agregado)
M1. Conexión cliente-servidor en Go entre equipos distintos (TCP o WebSocket).	**O1.** Canales o salas por área o por VLAN.
M2. Inicio de sesión con credenciales gestionadas por FastAPI; el servidor rechaza conexiones sin sesión válida.	**O2.** Indicador de usuario en línea / «escribiendo…».
M3. Envío y recepción de mensajes de texto entre usuarios autenticados.	**O3.** Cifrado del canal (TLS) entre cliente y servidor.
M4. Historial básico persistente y consultable.	**O4.** Envío de archivos pequeños.
M5. Control de sesiones: expiración y cierre de sesión.	**O5.** Panel sencillo de métricas para el administrador.
M6. Validación de entradas y manejo explícito de errores.	**O6.** Notificaciones de mensajes nuevos.
M7. Registro de métricas (latencia, tiempos, errores) para el componente estadístico.	

4.1 Responsabilidad de la autenticación: ¿quién registra a los usuarios?

El **registro de usuarios, el almacenamiento seguro de contraseñas (hash) y la emisión de la sesión/token** son responsabilidad exclusiva del **servicio FastAPI**, tal como lo establece el Proyecto Integrador. El servidor Go **no registra usuarios ni almacena contraseñas**: únicamente solicita a FastAPI iniciar sesión o validar un token y, según la respuesta, acepta o rechaza al cliente.

4.2 Fuera de alcance

Conforme a las exclusiones del proyecto: no se implementará alta disponibilidad, balanceo de carga, computación en la nube, SDN, IPv6 ni otros protocolos de enrutamiento distintos de OSPF; tampoco correo electrónico público ni transmisión profesional de video. El chat opera como servicio empresarial interno y controlado.

5. Requisitos y trazabilidad (componente POO)

Los requisitos RF-SW-01 a RF-SW-03 se adaptan al chat y se vinculan con las tareas del cronograma (sección 7) y con la semana en que se implementarán y demostrarán.

Código	Requisito adaptado al chat	Tareas	Implementación	Demostración / verificación
RF-SW-01	Un cliente ubicado en una sede consume la funcionalidad ejecutada por el servidor Go y recibe una respuesta válida.	4.1, 4.2	Fase 4 · semanas 8–10	Demo parcial en laboratorio: **semana 10**. Demo entre sedes: **semana 12**. Verificación formal: semana 13.
RF-SW-02	Enviar, recibir y consultar historial se completan sin errores críticos y la información persiste.	4.1, 4.2	Fase 4 · semanas 8–10	Demo parcial: **semana 10**. Pruebas funcionales: semanas 13–14.
RF-SW-03	El acceso al chat exige una sesión válida emitida por FastAPI; credenciales inválidas son rechazadas.	4.3, 5.1, 5.3	FastAPI: semanas 8–9. Integración Go ↔ FastAPI: semana 11.	**Demostración: semana 12** (evidencias de autenticación desde la app Go). Pruebas de credenciales: semana 13.

5.1 Dependencia clave entre tareas

Antes de integrar el chat con FastAPI (tarea 5.1, semana 11) deben estar terminadas dos tareas de la Fase 4: la **4.3** (servicio FastAPI local con registro, inicio de sesión y emisión de token, semanas 8–9) y la **4.1** (servidor Go que acepta conexiones y procesa mensajes, hasta la semana 10). Sin un servicio de autenticación funcional y un servidor operativo no es posible probar la integración. La dependencia está marcada en el cronograma.

6. Programación Orientada a Objetos aplicada en Go

Go no tiene clases ni herencia clásicas; los conceptos de POO se expresan así en el proyecto:

Concepto de POO	Cómo se expresa en Go	Aplicación en el chat
Clase / objeto	**struct** con **métodos** asociados.	Usuario, Mensaje, Sesion, ClienteChat, ServidorChat.
Encapsulamiento	Identificadores **exportados** (Mayúscula, visibles +) y **no exportados** (minúscula, privados −); constructores como NuevoUsuario().	Los campos de Sesion (token, expiraEn) son privados y se leen con métodos: Token(), EsValida().
Herencia	No existe; se sustituye por **composición** (un struct contiene a otros).	ServidorChat compone un Autenticador y un RepositorioMensajes; no hereda de ellos.
Polimorfismo	**Interfaces** satisfechas implícitamente.	La interface Autenticador la implementa AutenticadorFastAPI (y una versión simulada para pruebas).
Manejo de errores	Valor de retorno **error** explícito.	Todas las operaciones cliente-servidor devuelven error.

Concepto de POO	Cómo se expresa en Go	Aplicación en el chat
Modularidad y reutilización	**Paquetes** (internal/...).	Paquetes separados: red, lógica de negocio, modelo y control de usuarios.

Structs que representan los conceptos principales del dominio:

Struct	Campos (privados)	Métodos públicos	Paquete
Usuario	id, nombreUsuario, rol	ID(), NombreUsuario(), Rol(), EsAdmin()	internal/modelo
Mensaje	id, remitenteID, destinatarioID, contenido, enviadoEn	Contenido(), RemitenteID(), Validar()	internal/modelo
Sesion	token, usuarioID, expiraEn	Token(), UsuarioID(), EsValida()	internal/modelo

El diagrama de clases UML preliminar se entrega en el documento **04_Diagrama_UML_Clases** y se completará en la Fase 3.

7. Cronograma de trabajo del equipo

Cada tarea tiene su propia fila, con su duración en semanas y su responsable. Las barras se colorean por parcial. Las flechas rojas indican la dependencia entre las tareas 4.1 y 4.3 y la integración 5.1.



Responsables: S = Santiago Yambay · D = Daniel Villareal · E = Esteban Trujillo · T = todo el equipo

8. Declaración de integridad académica, licencia y uso de herramientas de IA

8.1 Integridad académica

Los integrantes del equipo declaramos que este proyecto es de nuestra autoría y que cada integrante comprende y puede explicar y defender todo el contenido entregado. Todo material de terceros (código, imágenes, librerías, textos) se identifica con su fuente y licencia en el README del repositorio. No se presentan trabajos ajenos como propios y las contribuciones individuales pueden verificarse mediante el historial de commits y el tablero de tareas.

8.2 Licencia

El repositorio usa de forma preliminar la licencia de código abierto **MIT**, por ser permisiva y compatible con las dependencias previstas (biblioteca estándar de Go, FastAPI y librerías de licencias permisivas). Los derechos de autor corresponden a los tres integrantes del equipo. La licencia podrá revisarse antes de la entrega definitiva; las librerías de terceros conservan sus propias licencias.

8.3 Uso de herramientas de inteligencia artificial

El equipo utilizó **Claude (Anthropic)** como asistente de apoyo durante la Fase 1, exclusivamente para: (a) organizar la estructura de los documentos, (b) redactar borradores que luego fueron revisados y ajustados por los integrantes, (c) revisar ortografía y coherencia, y (d) proponer un borrador del diagrama de clases y de la estructura del repositorio.

- Las decisiones del proyecto (elección del sistema, alcance, reparto de roles, acuerdos del contrato) fueron tomadas y validadas por el equipo.
- El contenido asistido fue verificado contra el documento del Proyecto Integrador; los integrantes responden por su exactitud.
- El uso de IA se registrará en el README y en los commits o pull requests correspondientes (etiqueta «asistido-por-ia»).
- La IA no sustituye la comprensión del equipo: cada integrante debe poder explicar su parte en la defensa.

Santiago Yambay	Daniel Villareal	Esteban Trujillo
_____ Firma	_____ Firma	_____ Firma

Fecha: 12 de octubre de 2026